

AI 技术调研与场景分析

课程：软件过程与管理
学院：四川大学软件学院
学期：2026 年春
作者：请填写姓名 / 学号 / 小组成员
提交格式：Markdown
提交渠道：GitHub

一、AI 技术应用场景匹配报告

1.1 调研背景

2024 年以来，AI 在软件工程中的应用从单点代码补全扩展到需求分析、架构设计、测试生成、代码审查、故障定位和智能运维等更广泛的工程环节。Stack Overflow 2024 开发者调查显示，大量开发者已经使用或计划使用 AI 工具参与开发流程，其中最主要的预期收益是提高生产率、提升效率和加快学习速度。DORA 2024 报告也指出，AI 能提升个人生产力、开发流动性和工作满意度，但同时可能对交付稳定性和吞吐量带来意外影响。这说明 AI 引入过程时不能只看“生成速度”，还必须关注质量门禁、可验证性、团队协作和软件交付稳定性。

对于课程作业中的软件过程改进而言，AI 技术的价值主要体现在三个层面。第一是知识密集型文档活动的辅助，例如需求访谈整理、用户故事生成、验收标准补全和会议纪要归纳。第二是规则密集型工程活动的辅助，例如代码审查、静态检查解释、测试用例生成、缺陷复现步骤整理。第三是数据密集型运行活动的辅助，例如日志聚类、异常检测、根因分析和容量预测。基于这三个层面，本文将原有过程中的高耗时、高重复、高风险环节作为优先改进对象。

1.2 AI 在软件工程各阶段的应用趋势

生命周期阶段	典型 AI 应用	工程价值	主要限制
需求分析	访谈转写摘要、用户故事生成、需求一致性检查、验收标准补全	降低需求文档初稿成本，提高需求覆盖度	容易遗漏业务上下文，需要产品经理和客户代表确认
系统设计	架构方案建议、UML 草图生成、接口契约草拟、设计模式推荐	提高方案探索速度，辅助形成设计备选项	不能替代架构决策，需验证非功能需求和技术约束
编码实现	代码生成、自动补全、重构建议、单元测试辅助	缩短样板代码和常规逻辑实现时间	存在幻觉、漏洞、依赖版本错误和风格不一致问题
代码审查	PR 预审、可读性建议、潜在缺陷提示、安全规则检查	在人工审查前发现低级问题，减轻审查负担	AI 审查不能作为合并批准，仍需人工审查
测试验证	测试用例生成、边界条件补全、测试脚本生成、缺陷预测	提升测试覆盖率和用例设计速度	需要执行验证，否则可能生成无效或重复用例
运维监控	日志聚类、异常检测、根因分析、容量预测	缩短故障定位时间，提高响应效率	需要高质量监控数据和告警规则校准

1.3 与原有软件过程的匹配原则

AI 场景匹配不能简单按照“哪里能用 AI 就在哪里加工具”的思路进行，而应满足以下原则。

第一，优先选择输入和输出都较清晰的环节。例如测试用例生成的输入是需求说明、接口文档和业务规则，输出是测试用例表、测试脚本或断言集合，便于评估质量。第二，优先选择人工重复劳动较高的环节，例如需求文档初稿、测试用例初稿、代码审查清单和日志摘要。第三，优先选择可以通过现有门禁验证的环节，例如生成的代码可以通过单元测试、静态扫描和人工审查验证，生成的测试用例可以通过需求覆盖矩阵和执行结果验证。第四，避免将 AI 放在高责任、不可逆、缺乏验证手段的决策点上，例如生产环境自动变更、数据库结构自动修改和安全策略自动放行。

1.4 关键场景识别

结合原有软件过程的活动、角色和交付物，本文识别出五个 AI 技术可显著提升效率的关键环节。

编号	关键环节	原有痛点	AI 介入方式	预期收益	适配优先级
S1	需求分析与用户故事生成	访谈材料分散，需求文档编写耗时，验收标准不完整	使用 LLM 根据访谈纪要生成用户故事、验收标准和需求冲突清单	文档初稿时间减少 30% 到 50%，需求遗漏率下降	高
S2	架构设计与接口契约草拟	设计备选方案不足，接口字段容易遗漏	使用 AI 根据需求和约束生成架构备选、接口草案和风险问题	提升方案比较充分性，减少接口返工	中
S3	编码实现与代码审查	常规代码重复，人工审查容易关注风格问题而忽略逻辑问题	使用 Copilot 类工具辅助编码，并在 PR 前进行 AI 预审	降低低级缺陷进入人工评审的概率	高
S4	测试用例生成与覆盖分析	测试人员编写用例耗时，边界条件和异常路径容易遗漏	使用 AI 根据需求、接口和历史缺陷生成测试用例，再由测试人员筛选执行	用例设计时间减少，边界覆盖率提升	高
S5	运维日志分析与根因定位	故障日志量大，定位依赖资深人员经验	使用日志聚类、异常检测和 LLM 摘要辅助定位根因	缩短平均恢复时间，形成故障知识库	中

1.5 场景匹配结论

本文选择 S1、S3、S4、S5 作为主要改进环节，其中 S4 “AI 辅助测试用例生成与覆盖分析” 作为原型验证场景。选择理由如下。

第一，测试阶段具有明确的输入和输出，便于量化比较。需求说明、接口定义、业务规则可以作为 AI 输入，输出的测试用例可以用覆盖率、有效率、重复率、缺陷发现数和人工修改量进行评价。第二，测试工作中存在大量重复劳动，适合使用 AI 生成初稿，再由测试人员审核、补充和执行。第三，测试验证本身就是质量门禁的一部分，AI 输出不会直接进入生产系统，风险相对可控。第四，原有过程通常已经包含测试计划、测试用例、缺陷报告和测试总结，AI 节点可以自然嵌入现有交付物体系。

参考文献与资料

1. DORA, *Accelerate State of DevOps Report 2024*, Google Cloud, 2024. <https://dora.dev/research/2024/dora-report/>
2. Stack Overflow, *2024 Developer Survey: AI*, 2024. <https://survey.stackoverflow.co/2024/ai/>
3. GitHub Docs, *Using GitHub Copilot code review*, 2026 年访问. <https://docs.github.com/en/copilot/how-tos/use-copilot-agents/request-a-code-review/use-code-review>
4. Haowei Cheng et al., *Generative AI for Requirements Engineering: A Systematic Literature Review*, arXiv:2409.06741, 2024. <https://arxiv.org/abs/2409.06741>
5. Junwei Liu et al., *Large Language Model-Based Agents for Software Engineering: A Survey*, arXiv:2409.02977, 2024. <https://arxiv.org/abs/2409.02977>
6. 中国信息通信研究院, 《智能化软件工程技术和应用要求 第 3 部分: 智能测试能力》, 2024。
7. GitHub Blog, *Survey reveals AI's impact on the developer experience*, 2024. <https://github.blog/news-insights/research/survey-reveals-ais-impact-on-the-developer-experience/>